

Lab 1.2 IoT Sensor Data Collection 1 - Temperature Sensor

1.1 Plotting the sensor data from the DS18B20

Plot the measured data from the CSV file using the template code below and attach the two plots (temperature according to timestamp).

Lets plot the temperature graph first.

Import essential libraries:

```
In [2]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import io
import seaborn as sns
```

Load CSV Data

The following code allows you to upload the data stored in a csv file to this Colab notebook.

```
In [ ]: #This code allows you to upload a file from your local drive
from google.colab import files
uploaded = files.upload()
#If you will work with excel, please save your file with a .CSV format
#Check the output file name below. You will use that file name in the next block.
```

```
In [4]: #Write the file name from the output above, inside the ' ' within the brackets.
#It will sub where it reads 'yourfilename.csv'
#If you want to check the uploaded file, click Files on the left-hand side menu
df = pd.read_csv("./L1_temp_log.csv")
df
```

Out[4]:

	Timestamp	Temperature (°C)	Temperature (°F)
0	2026-01-28 22:23:57	17.00	62.60
1	2026-01-28 22:23:59	17.06	62.71
2	2026-01-28 22:24:01	17.06	62.71
3	2026-01-28 22:24:02	17.06	62.71
4	2026-01-28 22:24:04	17.06	62.71
5	2026-01-28 22:24:06	17.06	62.71
6	2026-01-28 22:24:08	17.06	62.71
7	2026-01-28 22:24:09	17.06	62.71
8	2026-01-28 22:24:11	17.12	62.83
9	2026-01-28 22:24:13	17.25	63.05
10	2026-01-28 22:24:15	17.25	63.05
11	2026-01-28 22:24:17	17.31	63.16
12	2026-01-28 22:24:18	17.38	63.27
13	2026-01-28 22:24:20	17.44	63.39
14	2026-01-28 22:24:22	17.50	63.50
15	2026-01-28 22:24:24	17.56	63.61
16	2026-01-28 22:24:26	17.56	63.61
17	2026-01-28 22:24:27	17.62	63.73
18	2026-01-28 22:24:29	17.69	63.84
19	2026-01-28 22:24:31	17.69	63.84
20	2026-01-28 22:24:33	17.75	63.95
21	2026-01-28 22:24:35	17.81	64.06
22	2026-01-28 22:24:36	17.81	64.06
23	2026-01-28 22:24:38	17.81	64.06
24	2026-01-28 22:24:40	17.88	64.17
25	2026-01-28 22:24:42	17.94	64.29
26	2026-01-28 22:24:44	17.94	64.29
27	2026-01-28 22:24:45	18.00	64.40
28	2026-01-28 22:24:47	18.00	64.40
29	2026-01-28 22:24:49	18.06	64.51
30	2026-01-28 22:24:51	18.06	64.51
31	2026-01-28 22:24:53	18.12	64.62
32	2026-01-28 22:24:54	18.12	64.62
33	2026-01-28 22:24:56	18.12	64.62
34	2026-01-28 22:24:58	18.12	64.62
35	2026-01-28 22:25:00	18.19	64.74
36	2026-01-28 22:25:01	18.19	64.74
37	2026-01-28 22:25:03	18.19	64.74

	Timestamp	Temperature (°C)	Temperature (°F)
38	2026-01-28 22:25:05	18.19	64.74
39	2026-01-28 22:25:07	18.25	64.85
40	2026-01-28 22:25:09	18.25	64.85
41	2026-01-28 22:25:10	18.25	64.85
42	2026-01-28 22:25:12	18.25	64.85
43	2026-01-28 22:25:14	18.25	64.85
44	2026-01-28 22:25:16	18.31	64.96
45	2026-01-28 22:25:18	18.31	64.96
46	2026-01-28 22:25:19	18.31	64.96
47	2026-01-28 22:25:21	18.31	64.96
48	2026-01-28 22:25:23	18.31	64.96
49	2026-01-28 22:25:25	18.38	65.08
50	2026-01-28 22:25:27	18.31	64.96
51	2026-01-28 22:25:28	18.38	65.08
52	2026-01-28 22:25:30	18.38	65.08
53	2026-01-28 22:25:32	18.38	65.08
54	2026-01-28 22:25:34	18.38	65.08
55	2026-01-28 22:25:36	18.38	65.08
56	2026-01-28 22:25:37	18.38	65.08

Data plotting

Plot a chart that includes temperature (in °C, °F) in the code box below:

```
In [8]: df['time'] = pd.to_datetime(df["Timestamp"])
time = df['time'].dt.hour*3600 + df['time'].dt.minute*60 + df['time'].dt.second
t1 = time - time[0]
temperatureC = df['Temperature (°C)']

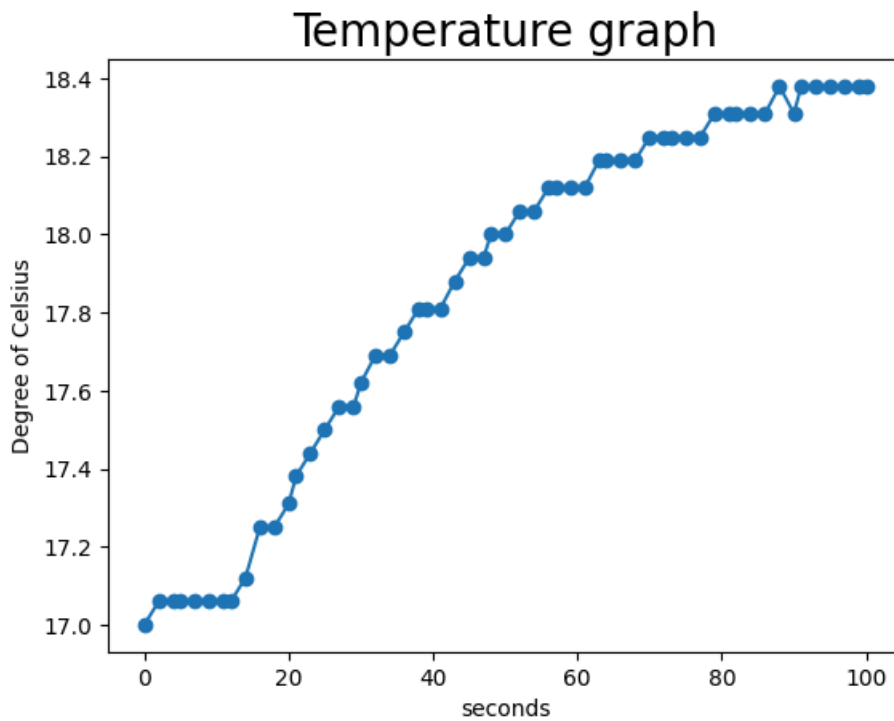
#Code section of the random number generator with plotting
fig = plt.figure() # Creates a new figure

ax = fig.add_subplot(111) # add a subplot to the new figure, 111 means "1x1 grid, first subplot"
#fig.subplots_adjust(top=0.80) # adjust the placing of subplot, adjust top, bottom, left and right spacing
ax.set_title('Temperature graph',fontsize= 20) # title of plot

ax.set_xlabel('seconds',fontsize = 10) #xlabel
ax.set_ylabel('Degree of Celsius', fontsize = 10) #ylabel

ax.plot(t1,temperatureC,'-o') #plotting the data with marker '-o'
#ax.axis([0, 10, 0, 10]) #specifying plot axes lengths
plt.show()

# prints the graph
```



Task 2.1

Plot a chart using that includes both temperature unit in the code box below:

```
In [11]: #Insert your coding below this line
df['time'] = pd.to_datetime(df["Timestamp"])
time = df['time'].dt.hour*3600 + df['time'].dt.minute*60 + df['time'].dt.second
t1 = time - time[0]
temperatureC = df['Temperature (°C)']
temperatureF = df['Temperature (°F)']

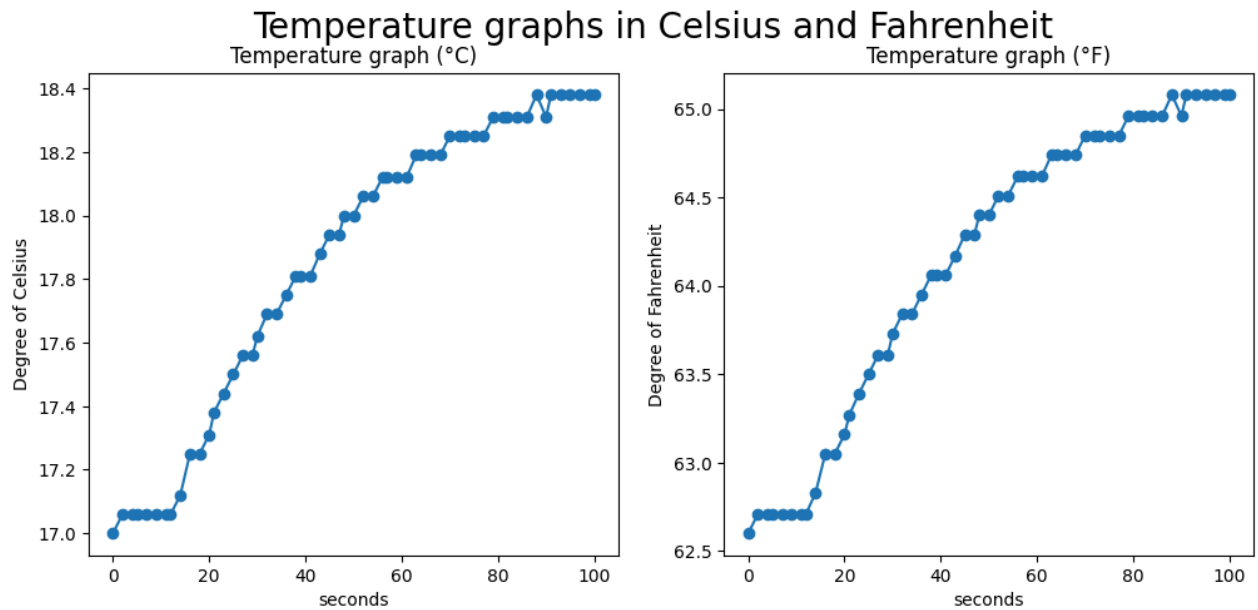
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5)) # Creates a new figure with 2 subplots side by side
fig.suptitle('Temperature graphs in Celsius and Fahrenheit', fontsize=20) # overall title for the figure

ax1.set_title('Temperature graph (°C)', fontsize= 12) # title of plot
ax1.set_xlabel('seconds', fontsize = 10) #xlabel
ax1.set_ylabel('Degree of Celsius', fontsize = 10) #ylabel

ax2.set_title('Temperature graph (°F)', fontsize= 12) # title of plot
ax2.set_xlabel('seconds', fontsize = 10) #xlabel
ax2.set_ylabel('Degree of Fahrenheit', fontsize = 10) #ylabel

ax1.plot(t1, temperatureC, '-o') #plotting the data with marker '-o'
ax2.plot(t1, temperatureF, '-o') #plotting the data with marker '-o'
#ax.axis([0, 10, 0, 10]) #specifying plot axes lengths
plt.show()

# prints the graph
```



Lab1 Summary and Deliverables

Answer the following questions for your achievements

Q1. Please summarize Lab1.

In Lab 1, we learned how to set up a raspberry pi to take input from a 1-wire sensor device using the GPIO pins. We also learned how the bus connector works and how to view the pinout to verify that we are using the correct pins to connect our sensors. Finally, we used how to communicate with the sensors using python and took temperature measurements. We also learned how to graph these temperature measurements using matplotlib pyplot.

Q2. What skills did you have to develop to accomplish this project?

I needed to develop better command line skills through bash, troubleshooting, and more python skills.

Q3. What aspects of this project were the most beneficial for your learning?

The most beneficial aspects of this project were learning more about the configuration of the raspberry pi and where settings are stored. Also, it was beneficial to learn how to communicate with external devices over the 1-wire GPIO pins.

Q4. What challenges did you encounter in completing the project?

Because of the winter weather and dry winter air inside my house, I believe I burned GPIO23 and GPIO24. This is my hypothesis because twice I tried to touch the thermocouple to warm it up and felt a bit of a shock and immediately the sensor stopped reporting data and ssh disconnected. I also had some difficulty transferring the files from the raspberry pi to my computer.

Q5. How did you overcome the challenges or remedy the problems encountered?

To overcome these challenges, I wrapped the thermocouple in a bit of anti-static material that was delivered with my raspberry pi. I also changed the boot config file to look at GPIO 22 instead of 23. To overcome the challenge of transferring files, I decided to use sftp from my computer instead of using WinSCP since I am more familiar with the commands for the sftp protocol.

Get back to [Lab Index Page](#)